

SVEUČILIŠTE U ZAGREBU
FAKULTET ELEKTROTEHNIKE I RAČUNARSTVA

Seminar 2

Primjena algoritma fastText na sustav ispravi.me

Nikša Brala

Zagreb, svibanj 2025.

Sadržaj

1. Uvod	3
2. Ugniježđeni vektori	4
3. FastText	7
3.1. Općenito o implementaciji	7
3.2. Primjena na sustav ispravi.me	8
4. Primjena algoritma fastText na sustav ispravi.me	9
4.1. Korpus za treniranje fastText modela	9
4.2. Postavke fastText modela	9
4.3. Konfiguracije modela	10
4.4. Očekivanja od modela	11
4.5. Kvalitativno vrednovanje istreniranih modela	11
4.5.1. Evaluacija nad pravilno napisanim riječima	11
4.5.2. Evaluacija nad pogrešno napisanim riječima	15
5. Zaključak	18
6. Literatura	20

1. Uvod

U suvremenom digitalnom društvu točnost i ispravnost jezika postaju ključne komponente svakodnevnih elektroničkih komunikacija. Automatizirano prepoznavanje i ispravljanje pravopisnih pogrešaka više nije samo tehnički izazov, već i temeljna potreba u kontekstu masovne produkcije i konzumacije pisanog sadržaja na internetu. Iako brojni spellchecking alati već postoje za velike jezike poput engleskog, u jezicima s manjim brojem govornika, kao što je hrvatski, ovakvi sustavi još uvijek često pate od ograničene pokrivenosti, loše semantičke osjetljivosti i nedostatka fleksibilnosti pri radu s pogreškama koje izlaze izvan okvira strogo definiranih leksičkih pravila.

Algoritamski pristupi koji se temelje na klasičnim rječnicima i pravilima imaju teškoća u razumijevanju konteksta i prepoznavanju fonetski sličnih, ali nepostojećih riječi, što značajno ograničava njihovu učinkovitost. U tom kontekstu, uvođenje metoda dubokog učenja i vektorskih reprezentacija riječi predstavlja novi smjer razvoja pametnijih jezičnih modela koji mogu nadomjestiti dio semantičkog razumijevanja u obradi prirodnog jezika.

Jedan od najistaknutijih algoritama u ovom području je fastText, alat za učenje vektorskih prikaza riječi koji koristi morfološke informacije kroz subwords i omogućuje učinkovitu obradu i rijetkih i nepostojećih riječi. Posebna prednost fastText-a leži u njegovoj sposobnosti da kreira reprezentacije i za riječi koje se nisu pojavile u trening korpusu, što ga čini posebno korisnim za zadatke ispravljanja pravopisnih pogrešaka gdje se često susrećemo s oblicima koje algoritam ne može pronaći u unaprijed definiranom rječniku.

U ovom se seminarskom radu istražuje mogućnost primjene fastText algoritma u svrhu unaprjeđenja postojećeg sustava za automatsko ispravljanje pravopisnih pogrešaka na hrvatskom jeziku – konkretno, ispravi.me. Rad ima višestruki cilj: prvo, detaljno analizirati kako različiti hiperparametri utječu na kvalitetu naučenih modela; drugo, istražiti semantičku osjetljivost modela kroz kvalitativne testove; i treće, evaluirati učinkovitost u kontekstu primjene ugniježđenih vektora implementacijom fastText na hrvatski jezik te na ispravljanje stvarnih pravopisnih pogrešaka. Kroz eksperimentalno treniranje šest modela na specifičnom korpusu hrvatskog jezika, provedeno je temeljito vrednovanje koje omogućuje donošenje zaključaka o prikladnosti fastText algoritma za integraciju u konkretne aplikacije obrade jezika poput ispravi.me.

2. Ugniježđeni vektori

Ugniježđeni vektori (engl. *word embeddings*) predstavljaju revolucionaran pristup pretvorbe riječi u numeričke reprezentacije koje zadržavaju semantičke i sintaktičke odnose. Za razliku od tradicionalnih metoda poput *one-hot* kodiranja, koje stvaraju rijetke vektore visoke dimenzionalnosti i gdje svaka riječ ima jedinstveni vektor s jednom jedinicom i ostalim nulama, moderni ugniježđeni vektori predstavljaju preslikavanje riječi u kontinuirani vektorski prostor R_d , tj. koriste gusto popunjene vektore niže dimenzije (d obično varira između 50 i 300) koji hvataju distribucijske svojstva riječi iz korpusa. Ovaj pristup rješava fundamentalni problem u računalnom razumijevanju jezika - kako pretvoriti simbolički, diskretni sustav kao što je ljudski jezik u numerički format koji računala mogu učinkovito procesirati i *razumjeti*.

Temeljni princip ovih reprezentacija formaliziran je distribucijskom hipotezom lingvistike (J.R. Firth: "Riječ je definirana kontekstom u kojem se pojavljuje"), koja se matematički modelira kroz uvjetne vjerojatnosti:

$$P(c \mid w) = \frac{\exp(\vec{v}_c \cdot \vec{v}_w)}{\sum_{c' \in V} \exp(\vec{v}_{c'} \cdot \vec{v}_w)}$$

gdje $\vec{v}_w$ i $\vec{v}_c$ predstavljaju vektorske reprezentacije ciljne i kontekstne riječi.

Pointwise Mutual Information (PMI) povezuje ove vjerojatnosti sa statistikom supojavljivanja riječi:

$$\text{PMI}(w, c) = \log \frac{P(w, c)}{P(w)P(c)} \approx \vec{v}_w \cdot \vec{v}_c$$

Distribucijsku je hipotezu lingvistike popularizirao J. R. Firth svojom izjavom "Reci mi s kim se družiš, reći ću ti tko si" primijenjenom na riječi. Ova hipoteza sugerira da riječi koje se pojavljuju u sličnim kontekstima imaju tendenciju imati slična značenja. Primjerice, riječi *automobil* i *vozilo* često se pojavljuju u sličnim sintaktičkim i semantičkim okvirima, te će njihovi vektori u embedding prostoru biti blizu jedan drugome. Tradicionalni su pristupi računalnoj lingvistici često zanemarivali ovu bogatu kontekstualnu informaciju, no vektorske je reprezentacije izravno kodiraju. Ovakav pristup omogućuje računalima da uče značenje riječi na

temelju njihovih kontekstualnih veza u velikim korpusima tekstova.

Formalno, za riječ w i kontekst C , vektorska reprezentacija mora zadovoljavati:

$$\vec{v}_w \cdot \vec{v}_c \propto \log P(w | c)$$

Među važne se matematičke temelje ubraja i pojam prostorne geometrijske interpretacije koji otkriva da semantički odnosi (npr. sinonimi, antonimi) poput analogija (npr. $\vec{v}_{\text{kralj}} - \vec{v}_{\text{muškarac}} + \vec{v}_{\text{žena}} \approx \vec{v}_{\text{kraljica}}$) odražavaju linearne transformacije u vektorskom prostoru. Ovo svojstvo proizlazi iz činjenice da se riječi s istim semantičkim ulogama (npr. imenice u genitivu) grupiraju u potprostore.

Ugniježđeni vektori, dakle, donose mnoge prednosti:

1. **Gusta reprezentacija:** za razliku od sparse prikaza ("one-hot"), ugniježđeni vektori koriste fiksne, relativno male dimenzije (npr. 100–300), što smanjuje memorijsku i računalnu složenost;
2. **Generalizacija:** slične riječi imaju slične vektore, što omogućuje prijenos znanja na riječi koje nisu viđene u treningu;
3. **Semantička aritmetika:** operacije nad vektorima mogu kodirati semantičke odnose (npr. rod, množina, vrijeme).

Među prednostima primjene ugniježđenih vektora posebno treba istaknuti prednosti primjene konkretno na hrvatski jezik:

1. **Morfološko bogatstvo:** hrvatski jezik ima ~20 padežnih oblika po imenici, što tradicionalne metode čini neefikasnim, a ugniježđeni vektori automatski hvataju veze između korijena i izvedenih oblika (npr. "pisati" → "pišem", "pisaše");
2. **Rukovanje nepoznatim riječima:** kroz mehanizme poput subword informacija, modeli mogu generirati reprezentacije za riječi koje nisu viđene tijekom treniranja.

Ugniježđeni se vektori koriste kao ulazni podaci u gotovo svim modernim NLP modelima: klasifikacija teksta, prepoznavanje entiteta, strojno prevođenje, analiza sentimenta i, u ovom konkretnom slučaju, prepoznavanje pravopisnih pogrešaka.

Za zadatke poput spellcheckinga, mogućnost mjerenja semantičke sličnosti između pogrešno napisane riječi i kandidata za ispravak predstavlja ključnu prednost ugniježđenih vektora pred tradicionalnim leksičkim pristupima (npr. Levenshteinova udaljenost).

Među ključne implementacije ugniježđenih vektora spadaju Word2Vec, GloVe i FastText. Njihove su osnovne karakteristike prikazane tablicom [Tablica 1](#).

Model	Način rada	Prednosti za hrvatski	Ograničenja
Word2Vec	Predviđanje konteksta	Brzina treniranja	Loše rješava OOV riječi
GloVe	Faktorizacija matrice ko-ponašanja	Globalna statistika	Nema subword informacije
FastText	Subword n-grami	Robustnost na morfološke varijante	Veća računska složenost

Tablica 1 Ugniježđeni vektori - implementacije

3. FastText

3.1. Općenito o implementaciji

Biblioteku *fastText* razvio je Facebook AI Research (FAIR) 2016. godine kao proširenje Word2Vec modela, s ciljem poboljšanja rada s rijetkim riječima i morfološki bogatim jezicima – što uključuje i hrvatski jezik.

Glavna razlika između *fastTexta* i ranijih modela (poput Word2Vec-a) jest u načinu na koji se riječi predstavljaju. Umjesto da se svakoj riječi dodjeljuje jedinstveni vektor, *fastText* modelira riječi kao zbroj *n*-grama znakova (podriječi):

$$\vec{v}_{\text{riječ}} = \frac{1}{|G_w|} \sum_{g \in G_w} \vec{v}_g$$

gdje G_w predstavlja skup *n*-grama riječi *w* (npr. riječ "kuća": <ku, kuć, uća, ća>; ili za riječ "pisač": <pi, pis, isa, sač, ač>).

Svakom se *n*-gramu dodjeljuje vektor, a vektor riječi je suma svih njegovih *n*-grama.

Ovakav pristup omogućuje:

1. **Generalizaciju na nepoznate riječi:** ako se riječ ne nalazi u vokabularu, ali se njezini *n*-grami pojavljuju u korpusu, *fastText* može svejedno proizvesti smislen vektor (hrpa hashova (bucket hashing)) koristi se za mapiranje *n*-grama u fiksni broj pretinaca (obično 2M), što smanjuje memorijske zahtjeve, a modelu omogućuje da generira vektore čak i za neviđene riječi (npr. "pjevanj" → "pjevanje") analizom zajedničkih morfema;
2. **Bolje rukovanje fleksijama i tvorbom riječi:** hrvatski je jezik bogat deklinacijama i konjugacijama, pa modeliranje podriječi omogućuje prepoznavanje da su riječi "učitelj", "učitelja", "učiteljici" semantički bliske;
3. **Detekciju tipfelera i morfoloških varijanti:** analizom zajedničkih *n*-grama (npr. "pjevanje" i "pjevanj" dijele *n*-gram "pjev"), model može prepoznati pogrešno napisane riječi.

FastText koristi istu arhitekturu kao Word2Vec (skip-gram s negative sampling), ali vektor ciljne riječi konstruira kao sumu vektora svih njezinih n-grama. Funkcija cilja modela tada postaje:

$$\log \sigma(v'_{w_O}{}^T v_{w_I}) + \sum_{i=1}^k \mathbb{E}_{w_i \sim P_n(w)} [\log \sigma(-v'_{w_i}{}^T v_{w_I})]$$

gdje je σ sigmoid funkcija, a K broj negativnih uzoraka.

3.2. Primjena na sustav ispravi.me

Potencijalna primjena implementacije fastText na hrvatski spellchecker, tj. platformu ispravi.me donosi visoki potencijal za unaprjeđenje i modernizaciju postojećih funkcionalnosti sustava, od kojih se najviše ističu:

1. **Detekcija tipfelera:** FastText prepoznaje morfološke varijante kroz zajedničke n-grame (npr. "prijatelj" vs. "prijatel" imaju zajedničke n-grame "prij", "jat"), a eksperimenti na drugim jezicima pokazuju poboljšanje od 12% u odnosu na rule-based sustave;
2. **Generiranje prijedloga:** vektorska sličnost u prostoru omogućuje pronalaženje kandidata (npr. za "frend" → "prijatelj" preko n-grama "ren");
3. **Evaluacijske metrike i rangiranje prijedloga:** Mean Reciprocal Rank (MRR) mjeri kvalitetu prijedloga na temelju pozicije točnog odgovora u rang-listi, a Recall@k daje broj točnih prijedloga unutar prvih k rezultata.

Mogućnosti za poboljšanjima sustava sa sobom, s druge strane, donose i brojne nove izazove za sustav, od kojih su najznačajniji:

1. **Računska zahtjevnost:** subword pristup povećava broj parametara za ~30%;
2. **Pristranost u podacima:** modeli mogu naslijediti stereotipe iz korpusa (npr. rodne asocijacije);
3. **Integracija s postojećim sustavom:** potrebna je iscrpna modifikacija tokenizatora za podršku subword analize.

4. Primjena algoritma fastText na sustav ispravi.me

4.1. Korpus za treniranje fastText modela

Za potrebe je ovog rada trenirano ukupno šest različitih fastText modela na temelju korpusa tekstova prikupljenih u okviru platforme ispravi.me u razdoblju 2024. i 2025. godine. Prije samog treniranja, korpus je temeljito očišćen i normaliziran kako bi se uklonili svi elementi koji bi mogli negativno utjecati na kvalitetu naučenih vektorskih reprezentacija kao što su nepotrebni razmaci, rečenični znakovi, brojevi (znamenke) i specijalni znakovi. Naglasak je u okviru fastTexta stavljen na semantiku, pa je sav tekst korpusa prebačen u isključivo mala slova, tako da se izbjegnu semantički šumovi između istovjetnih pojava među kojima su neke početne riječi u rečenici, pa počinju velikim slovom.

Očišćeni korpus sadrži oko 33 milijuna riječi, što čini dovoljno veliku količinu podataka za treniranje kvalitetnih jezičnih reprezentacija uz primjenu algoritma skip-gram, kakav fastText koristi u svojoj implementaciji.

4.2. Postavke fastText modela

Treniranje fastText modela provedeno je s pomoću službene Python biblioteke *fasttext*, koja omogućava rad s riječima i njihovim podriječima (subwords), čime se postiže bolja robusnost prema morfološkim varijacijama i rijetkim riječima – što je iznimno važno za hrvatski jezik, koji se odlikuje visokom razinom morfološke kompleksnosti.

Prilikom treniranja modela, modeli su određeni kao eksperimenti s različitim kombinacijama ključnih hiperparametara koji imaju značajan utjecaj na ponašanje i performanse naučenih vektora. Pod ključne varirane parametre spadaju:

- **model:** tip modela (u ovom slučaju svi modeli su trenirani kao *skip-gram*, budući da se pokazao boljim za problemske domene s niskim frekvencijama riječi);
- **dim:** dimenzionalnost naučenih vektora (npr. 100 ili 300);
- **ws** (window size): kontekstni prozor koji određuje koliko se susjednih riječi uzima u obzir prilikom učenja reprezentacije;
- **epoch:** broj prolazaka kroz cijeli korpus tijekom treniranja;

- **minCount**: minimalna učestalost riječi koja mora biti zadovoljena da bi se riječ uključila u vokabular.

4.3. Konfiguracije modela

Ukupno je trenirano šest modela, od kojih svaki koristi drugačiju konfiguraciju parametara kako bi se omogućila usporedba različitih pristupa i bolje razumijevanje učinka svakog pojedinog hiperparametra. Konfiguracije i njihovi parametri opisani su u tablici [Tablica 2](#).

Model	Dimenzija(dim)	Window size (ws)	Epochs (epoch)	Min Count (minCount)	Napomena
1	100	5	5	5	Bazni model
2	100	5	10	5	Ispitivanje učinka većeg broja epoha
3	100	10	5	5	Veći kontekstni prozor
4	300	5	5	5	Veća dimenzionalnost vektora
5	300	10	5	5	Kombinacija većeg konteksta i dimenzije
6	300	10	10	2	Maksimalistička konfiguracija

Tablica 2 Konfiguracije korištenih modela

Kao što se vidi iz gornje tablice [Tablica 2](#), prvi model korišten je kao referentna točka, dok su ostali modeli sistematski uveli promjene u pojedinačne ili kombinirane parametre kako bi se analizirali njihovi efekti.

Primjerice, drugi model zadržava sve parametre baznog modela, ali udvostručuje broj epoha, što bi trebalo rezultirati boljim uvidom u dugoročne obrasce u podacima, iako uz potencijalno veću razinu prenaučivosti.

Treći model koristi prošireni kontekstni prozor, što može pomoći pri učenju semantički šire povezanih riječi.

Četvrti i peti model eksperimentiraju s većom dimenzionalnošću vektora (300), što omogućava reprezentaciju složenijih značenjskih nijansi riječi. Napokon, šesti model kombinira

sve prethodno navedene modifikacije, uključujući i smanjeni prag učestalosti riječi (`minCount = 2`), kako bi se omogućilo učenje i nad rijetkim riječima – što je ključno za primjenu u sustavima poput `ispravi.me`, gdje je pojava pogrešno napisanih ili rijetko korištenih riječi itekako česta.

4.4. Očekivanja od modela

Od modela se očekuje da će veća dimenzionalnost (modeli 4, 5 i 6) omogućiti učenje bogatijih i semantički preciznijih reprezentacija riječi, osobito za riječi s višeznačnim ili kompleksnim značenjem. Također, prošireni kontekst (modeli 3, 5 i 6) može pomoći u boljoj semantičkoj povezanosti između riječi koje nisu nužno lokalne (tj. neposredno susjedne).

Modeli s većim brojem epoha (2 i 6) mogu dublje „usvojiti” obrasce iz korpusa, ali su i podložniji prenaučenosti ako su podaci repetitivni ili ako model previše nauči "napamet".

S druge strane, spuštanje praga učestalosti riječi (`minCount=2` u modelu 6) omogućuje obuhvat šireg vokabulara, uključujući i rijetke riječi, što je osobito korisno za aplikacije provjere pravopisa, no može povećati šum u vektorskom prostoru ako su te riječi nedovoljno reprezentirane u korpusu.

4.5. Kvalitativno vrednovanje istreniranih modela

Nakon treniranja šest `fastText` modela nad očišćenim korpusom teksta, pristupili smo kvalitativnoj evaluaciji njihovih rezultata. Cilj ove faze istraživanja bio je analizirati kako različite konfiguracije hiperparametara utječu na semantičku osjetljivost i preciznost modela u kontekstu hrvatskog jezika. Evaluacija je provedena u dva smjera:

1. Nad skupom ispravno napisanih riječi uobičajenih u svakodnevnom jeziku;
2. Nad skupom pogrešno napisanih riječi karakterističnih za korisničke pogreške u pisanju.

Ova dvostruka analiza omogućila je uvid ne samo u lingvističku osjetljivost modela, nego i u njihovu primjenjivost u kontekstu automatskog ispravljanja pogrešaka — konkretno za platformu poput `ispravi.me`.

4.5.1. Evaluacija nad pravilno napisanim riječima

Riječi korištene u ovom dijelu evaluacije bile su podijeljene u pet semantičkih skupina: učestale imenice, glagoli, relativno rijetke riječi, dijalektalne riječi i apstraktni pojmovi. Za svaku

su riječ funkcijom *get_nearest_neighbors(word, k)* (gdje je *k* varijabilni broj susjeda koje funkcija treba generirati) dohvaćeni njezini najbliži susjedi (nearest neighbors) po svakom modelu, a sličnosti su mjerene kosinusnom metričkom mjerom. Rezultati su uspoređivani unutar pojedinih riječi, po svim modelima, kako bi se omogućila horizontalna usporedba performansi.

Rezultati su evaluacije imenice *škola* (kao primjera riječi iz semantičke skupine *učestale imenice*) prikazani tablicom [Tablica 3](#), a silazno su sortirani po indeksu podudarnosti.

Model					
1	2	3	4	5	6
škole	škole	školarka	školarka	škole	škole
gimnazija	školarka	predškola	školarca	učenika	učenika
pedagoška	učenika	autoškola	školarme	učenike	učenička
eškole	školarca	školarca	autoškola	učenička	učenici
odgojnoobrazovna	strukovna	školarina	školarina	razredna	učenike
učenika	školama	školica	školarci	gimnazija	jednosmjenska
razredna	gimnazija	školestva	predškola	odgojnoobrazovna	ekoškola
učenica	školska	školska	školarinu	učionica	učionica
strukovna	strukovnih	učenica	školaraca	razrednih	izvannastavna
strukovne	školarme	školarme	školarine	nastavu	strukovnih

Tablica 3 Evaluacija imenice "škola" po modelima

Rezultati su evaluacije glagola *pisati* (kao primjera riječi iz semantičke skupine *učestale imenice*) prikazani tablicom [Tablica 4](#), a silazno su sortirani po indeksu podudarnosti.

Model					
1	2	3	4	5	6
čitati	pisat	napisati	zapisati	čitati	čitati
napisati	napisati	zapisati	napisati	čitaćim	pisat
zapisivati	ispisati	ispisati	ispisati	napisati	napisati
ispisati	zapisati	isčitati	prepisati	pročitati	zapisati
zapisati	čitati	čitati	opisati	pisanjaraditi	zapisivati
pročitati	prepisati	prepisati	pripisati	pisat	govoriti
pisat	pisaći	pročitati	otpisati	zapisati	ispisati
recitirati	napisat	spočitati	popisati	zapisivati	pisanjaraditi
opisivati	upisati	učitati	upisati	objavljivati	prestavljati
dopisivati	zapisivati	ispisivati	raspisati	pripovijedati	napisat

Tablica 4 Evaluacija glagola "pisati" po modelima

Model					
1	2	3	4	5	6
trezora	trezora	reznor	trevor	trezora	trezora
trezorski	trezoru	trevor	trezeguet	trezorice	trezorice
trezoru	trezorski	trezeguet	reznor	trezorski	trezoru
trezorske	trezorske	tresać	tresor	trezorima	trezorski
trezorskih	trezorima	trek	tresać	trezoru	trezorima
trezorima	trezorskih	tremor	trejd	trezoraca	trezoraca
obveznice	euroobveznica	tresak	tremor	trezorskih	trezorske
obveznicu	obveznice	trzaj	trek	trezorske	trezorskih
obveznica	obveznicu	skor	trezven	obveznica	obveznicu
euroobveznica	obveznica	kor	tresak	obveznicu	obveznice

Tablica 5 Evaluacija riječi "trezor" po modelima

Rezultati su evaluacije riječi *trezor* (kao primjera riječi iz semantičke skupine *relativno rijetke riječi*) prikazani tablicom [Tablica 5](#), a silazno su sortirani po indeksu podudarnosti.

Model					
1	2	3	4	5	6
pomidore	pomidore	poor	poor	pomidore	pomidora
sezam	termidore	fjodor	poms	pomidora	pomidore
kotlet	izidor	isidor	pomican	zamirišu	kačkavalja
odreska	isidor	polydor	pomidore	kačkavalja	termidore
kunpir	pomodoro	teodor	pomirdbu	balsamico	vlasac
pirjaj	kolutiće	por	pomper	umido	balsamico
paradajz	sezam	prodor	polydor	pelate	umido
naribani	fjodor	izidor	pomirba	zamiriši	cvjetačom
omegol	inćune	zazor	pompidou	sjeckane	balsamica
poriluk	miñen	ponor	pomiri	samelje	balsamicom

Tablica 6 Evaluacija riječi "pomidor" po modelima

Rezultati su evaluacije riječi *pomidor* (kao primjera riječi iz semantičke skupine *dijalektalne riječi*) prikazani tablicom [Tablica 6](#), a silazno su sortirani po indeksu podudarnosti.

Model					
1	2	3	4	5	6
sposobnost	korisnost	nadmoć	nadmoć	superiornost	prirpvake
superiornost	premoć	mogućnost	nemoć	sposobnost	moću
stranost	nemoć	sposobnost	mogućnost	suverenost	stranost
ubojitost	stranost	moralnost	moćnu	odlučnost	uvjerenost
vitalnost	nadmoć	mudrost	moćniju	uvjerenost	sposobnost
istinoljubivost	prirpvake	odlučnost	premoć	korumpiranost	kupovnu
korisnost	genijalnost	besmislenost	mudrost	snagu	korisnost
slabost	vitalnost	vjernost	samopomoć	neustrašivost	suverenost
podređenost	istinoljubivost	bezvoljnost	svemoćnost	vitalnost	efikasnost
svjesnost	sporost	zanost	moralnost	toliku	muževnost

Tablica 7 Evaluacija riječi "moć" po modelima

Rezultati su evaluacije riječi *moć* (kao primjera riječi iz semantičke skupine *apstraktne riječi*) prikazani tablicom [Tablica 7](#), a silazno su sortirani po indeksu podudarnosti.

Analizom rezultata za odabrane reprezentativne riječi iz svake semantičke skupine uočen je jasan utjecaj hiperparametara na kvalitetu predloženih srodnih pojmova.

Za riječ *škola*, gotovo svi modeli dosljedno su vraćali visoko relevantne pojmove poput *učitelj*, *nastava* i *obrazovanje*, pri čemu su modeli s većim brojem dimenzija (posebice Model 4) pokazali nešto veću semantičku preciznost.

U slučaju glagola *pisati*, modeli s višim brojem epoha bolje su hvatali kontekstualne varijante kao što su *zapisivati*, *pisanje* i *autorski*; dok su modeli s manjim dimenzijama i prozorima (window size) davali manje precizne i šumovitije rezultate.

Za riječ *trezor*, koja pripada kategoriji relativno rijetko korištenih pojmova, značajna razlika među modelima bila je uopće u sposobnosti prepoznavanja srodnih termina – modeli s nižim minCount pragom bolje su uključili relevantne varijante.

Dijalektalna riječ *pomidor* bila je izazov za sve modele, no samo su se među rezultatima modela 1, 5 i 6 pojavili neki (relativno) suvisli rezultati (npr. *pelati* i *vlasac*), dok su ostali modeli uglavnom vraćali manje relevantne ili potpuno irelevantne prijedloge.

Konačno, kod apstraktne riječi *moć*, uočena je veća varijabilnost među modelima; dok su napredniji modeli dosljedno predlagali pojmove poput *mudrost* i *sposobnost*, jednostavniji modeli su često davali manje precizne ili generičke pojmove.

Ovi rezultati potvrđuju da semantička koherencija i osjetljivost modela ovise o pravilno odabranim konfiguracijama, osobito u slučajevima složenih, višeznačnih i rjeđe korištenih riječi.

4.5.2. Evaluacija nad pogrešno napisanim riječima

Za ovu je evaluaciju korišten skup od 10 čestih pravopisnih pogrešaka (npr. *riješenje*, *moć* i sl.), a cilj je bio testirati sposobnost modela da predloži ispravan oblik kao najbližeg susjeda. Ova metoda je posebno važna jer simulira konkretan slučaj upotrebe fastText modela u alatu za provjeru pravopisa. U daljnjem će se tekstu prikazati rezultati kvalitativne evaluacije nad primjerima učestalih pravopisnih pogrešaka.

Model					
1	2	3	4	5	6
rješenje	rješenje	polurješenje	polurješenje	rješenje	riješenjem
polurješenje	polurješenje	razrješenje	rješenje	riješenja	riješenja
riješenja	riješenja	rješenje	razrješenje	riješenjem	riješenju
riješenju	riješenju	rešenje	rešenje	riješenju	rješenje
riješeno	riješen	riješavanje	riješenja	rioješenje	rioješenje
rešenje	riješeno	zagušenje	riješe	polurješenje	riješenu
riješen	riješena	vršenje	riješen	riješiti	riješen
razrješenje	riješeni	rješavanje	riješenju	riješavanje	rešenje
riješe	riješenu	opravdanje	riješene	riješeno	polurješenje
riješavanje	rešenje	iskušenje	riješenu	Riješi	riješeno

Tablica 8 Evaluacija riječi "riješenje" po modelima

Rezultati su evaluacije riječi *riješenje* (kao primjera pogrešno napisane riječi s tipom pogreške *ije/je*) prikazani tablicom [Tablica 8](#), a silazno su sortirani po indeksu podudarnosti.

Model					
1	2	3	4	5	6
močibob	močibob	moët	moët	moči	moči
moči	moči	moi	moe	močibob	močibob
ruparanch	močvari	moe	moi	faal	močić
ploszek	močvarno	moš	moc	močić	močilama
puertas	močvaru	moc	moš	ruparanch	močvarno
pluton	močvara	možwe	mou	nč	močvarna
marsovac	močvare	mop	mop	sered	močvaru
lukacevic	humeljak	mou	moo	seidel	močvari
popes	peñon	mort	moa	močilama	močile
zoljan	močvarne	morro	možwe	sundin	močinić

Tablica 9 Evaluacija riječi "moč" po modelima

Rezultati su evaluacije riječi *moč* (kao primjera pogrešno napisane riječi s tipom pogreške *č/ć*) prikazani tablicom [Tablica 9](#), a silazno su sortirani po indeksu podudarnosti.

Rezultati evaluacije na pogrešno napisanim riječima *riješenje* i *moč* ukazuju na djelomičnu uspješnost fastText modela u detekciji i predlaganju ispravnih oblika.

Kod riječi *riješenje*, koja predstavlja klasičnu pravopisnu pogrešku zamjene *ije* s *je*, većina modela – osobito oni trenirani s većim brojem epoha i višim dimenzijama – uspjela je identificirati ispravnu varijantu *rješenje* kao najbližeg susjeda ili među prvih nekoliko prijedloga. Ovo potvrđuje da fastText uspješno koristi subword informacije za fonetski i morfološki bliske oblike.

S druge strane, za riječ *moč*, koja sadrži čestu pogrešku zamjene slova *ć* s *č*, nijedan od modela nije vratio ispravnu riječ *moć* među najbližim susjedima, što ukazuje na ograničenja modela u razlikovanju fonetski sličnih, ali semantički razdvojenih jedinica koje se razlikuju jednim slovom.

Ovi rezultati sugeriraju da fastText bolje generalizira nad pogreškama koje zahvaćaju dulje slogove i pravopisne sklopove (*ije/je*), dok ima ograničenu preciznost kod suptilnih ortografskih razlika koje se svode na jedan znak. Time se potvrđuje potreba za dodatnim slojevima filtriranja ili kombinacijom s leksičkim bazama kako bi se povećala točnost u prepoznavanju pravopisnih pogrešaka koje fastText samostalno teško razrješava.

5. Zaključak

U ovom je radu istražen potencijal primjene algoritma fastText na zadatak automatskog ispravljanja pravopisnih pogrešaka u hrvatskom jeziku. Poseban fokus stavljen je na evaluaciju mogućnosti njegove integracije u postojeći sustav ispravi.me, koji već dugi niz godina pruža podršku u pisanju ispravnog hrvatskog jezika. Kroz postupak treniranja šest modela različitih konfiguracija te njihovu kvalitativnu analizu, dobiven je vrijedan uvid u ponašanje fastText modela u jezičnom okruženju koje karakterizira bogata morfologija, fleksibilan red riječi i relativno ograničena količina digitalno dostupnih podataka.

Analiza je pokazala da izbor hiperparametara značajno utječe na performanse modela. Modeli trenirani s većim dimenzijama, dužim brojem epoha i širim kontekstnim prozorom pokazuju veću semantičku osjetljivost i bolju sposobnost prepoznavanja kontekstualno srodnih riječi, uključujući i fonetski slične varijante pogrešno napisanih riječi. U kvalitativnim testovima nad ispravno napisanim riječima, takvi modeli dosljedno predlažu ispravne oblike kao najbliže susjede, što pokazuje njihovu korisnost za zadatak spellcheckinga.

Međutim, rezultati također pokazuju da ništa nije jednoznačno i da uvođenje fastText algoritma sa sobom donosi određene rizike i izazove.

Prvo, treniranje modela zahtijeva veliku količinu kvalitetnog i očišćenog teksta, čije prikupljanje i priprema predstavljaju značajan resursni trošak, a postavlja se i pitanje što uopće znači *kvalitetan tekst* i koje su neke osnovne postavke koje kvalitetan tekst treba zadovoljiti.

Drugo, iako fastText dobro funkcionira s ispravnim oblicima riječi koje su fonetski i semantički bliske ispravnima, njegova preciznost opada u slučajevima višeznačnih riječi, dijalekata, kontekstualno dvosmislenih izraza i pogrešno napisanih riječi. To dovodi u pitanje isplativost primjene ove implementacije na ispravljanje pogrešaka u strojnom provjerniku pravopisa, jer su prikazani modeli relativno neuspješni pri ispravljanju pogrešaka, ali bi njihova primjena mogla biti itekako korisna i uspješna u kontekstu

predlaganja semantički sličnih riječi.

Treće, integracija u postojeći sustav zahtijeva pažljivo biranje kriterija za rangiranje i filtriranje prijedloga, kako bi se spriječilo unošenje pogrešaka temeljenih na vektorskoj sličnosti koja nije semantički utemeljena.

Unatoč tim ograničenjima, rezultati ovog rada ukazuju na to da fastText predstavlja vrlo obećavajući alat za proširenje funkcionalnosti i inteligencije alata poput ispravi.me. Njegova sposobnost generalizacije i rada s podriječima (subwords) omogućuje mu prednost u jezicima poput hrvatskog, gdje deklinacija, konjugacija i sufiksacija igraju ključnu ulogu u pisanju.

Konačno, iako je fastText samo jedan od mogućih algoritama vektorizacije riječi, ovo istraživanje potvrđuje da pravilnim odabirom parametara i kvalitetnom pripremom podataka možemo dobiti modele koji uvelike doprinose točnosti i korisničkoj vrijednosti sustava za strojnu provjeru pravopisa. Buduće istraživanje moglo bi uključivati usporednu analizu s novijim arhitekturama poput BERT-a ili kombinaciju fastText ugniježđenih vektora s leksičkim rječnicima, kako bi se postigao još viši stupanj preciznosti u prepoznavanju i ispravljanju pogrešaka.

6. Literatura

1. **Daniel Jurafsky, James H. Martin** (2024.) *Speech and Language Processing*
2. **Gordan Gledec, Šandor Dembitz** (2024.) *Razvoj i održavanje n-gramskog sustava hrvatskoga jezika*
3. **Experimental Comparison of Pre-Trained Word Embedding Vectors** (2022.) *Advances in Science and Technology Research Journal*. [Link](#)
4. **Nurdin, Z. A., Bernadus, Anugrayani** (2020.) *Performance Comparison of Word2vec, Glove, and Fasttext Embedding in Text Classification*. [Link](#)
5. **Gensim Documentation: Word2vec** (2023.) [Link](#)
6. **Alpha Quantum Blog** (2023.) *Introduction to Word Embeddings – Word2Vec, Glove, FastText and ELMo*. [Link](#)
7. **VITEZ University Repository** (2023.) *Primjena Bayesove vjerojatnosti u strojnom učenju* [Link](#)
8. **Mikolov, T. et al.** (2013.) *Efficient Estimation of Word Representations in Vector Space*. arXiv:1301.3781.
9. **Pennington, J. et al.** (2014.) *GloVe: Global Vectors for Word Representation*. Proceedings of EMNLP.
10. **Bojanowski, P. et al.** (2017.) *Enriching Word Vectors with Subword Information*. Transactions of the ACL.
11. **Joulin, A., Grave, E., Bojanowski, P., & Mikolov, T.** (2017.) *Bag of Tricks for Efficient Text Classification*. Proceedings of the 15th Conference of the European Chapter of the ACL (EACL 2017). [Link](#)
12. **fastText** – Library for efficient text classification and representation. Facebook AI Research (FAIR). [Link](#)
13. **Gyllenstein, A. C., & Sahlgren, M.** (2018.) *Embedding-based Spelling Correction*. Proceedings of the 27th International Conference on Computational Linguistics (COLING 2018). [Link](#)